

ChoreMate

Scalability Experiments Report

Load Testing · Notification Architecture · Async LLM Offloading

Experiment 1 Database Bottleneck SQLite → AWS Postgres 50 / 100 / 200 users	Experiment 2 Notification Layer Polling vs SSE 50 / 100 / 150 / 1000 burst	Experiment 3 Async AI Offloading Celery Workers 1 / 2 / 4 LLM queue on AWS
---	--	--

ChoreMate is a production multi-tenant household chore management platform using a FastAPI backend, SQLite/PostgreSQL data layer, Celery+Redis/SQS task queue, and a Gemini LLM AI chatbot. Three architectural scalability hypotheses were tested under graduated *Locust* synthetic load to quantify bottlenecks and validate solutions.

Key Findings at a Glance

Experiment	Baseline Problem	Mitigation	Result
1 · DB Backend	SQLite p95 = 17 ms (100 users)	AWS RDS Postgres (multi-process Gunicorn)	0 failures, p95 = 130 ms; tail spikes at 200 users
2 · Notifications	Polling: 10 s latency at 1000-burst	Redis Pub/Sub + SSE push	Polling server RPS 20–40× higher than SSE
3 · Async LLM	1-worker p50 = 13 s LLM delay	Celery workers scale-out (1→2→4)	4-worker p50 = 4.2 s (3.1× speedup, no failures)

Methodology: All experiments used Locust with pre-seeded users. Local experiments ran on a MacBook Air M2 (8 GB). Cloud experiments ran on AWS: EC2 t3.small app server, RDS t3.micro Postgres, ElastiCache Redis, SQS Celery workers, and an Application Load Balancer. Metrics: p50/p95/p99 latency, requests/s, failure rate, and custom event timers.

April 2026

Experiment 1 — Database Bottleneck

PURPOSE

Measure how the database backend limits API concurrency. Compare SQLite (single-writer, no concurrency) running locally against AWS RDS Postgres with full WAL concurrency under identical API load.

TRADE-OFF EXPLORED

SQLite offers zero operational overhead (single file) but serialises every write. Postgres adds network round-trip latency (~30–50 ms) but allows concurrent reads/writes via MVCC, enabling horizontal API scaling.

LIMITATIONS

Local SQLite results share CPU with the test driver; network is absent. The AWS instance (t3.micro RDS) cannot represent a production-grade Postgres deployment. Only read-heavy chore-listing endpoints were tested; write-intensive paths (chore assignment) were excluded from this run.

Experiment 1 — Database Backend Comparison

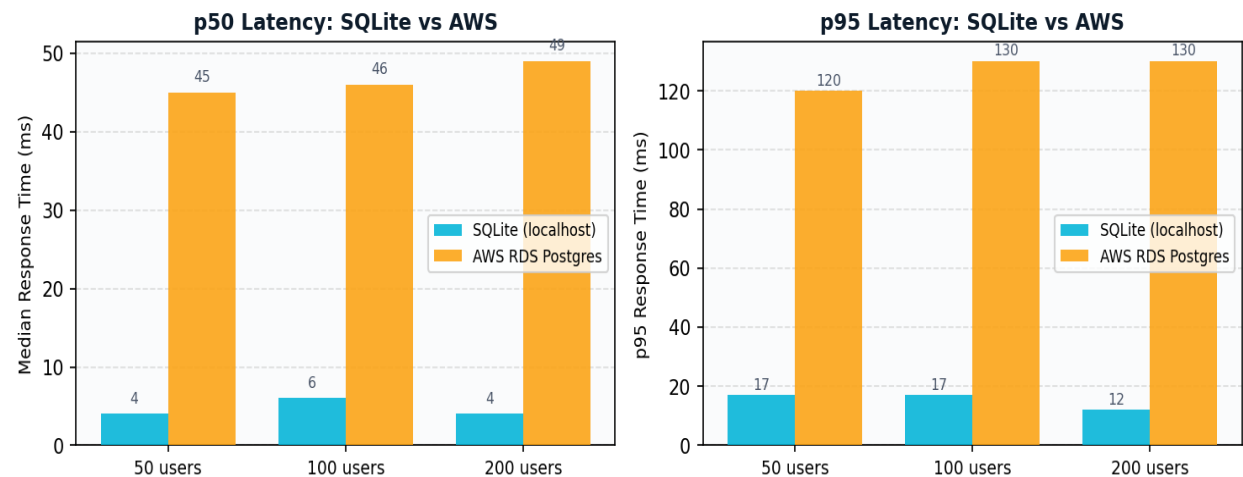


Figure 1.1 · p50 and p95 response times across endpoints at 50/100/200 concurrent users. SQLite (localhost) benefits from in-process DB access; AWS figures include network overhead.

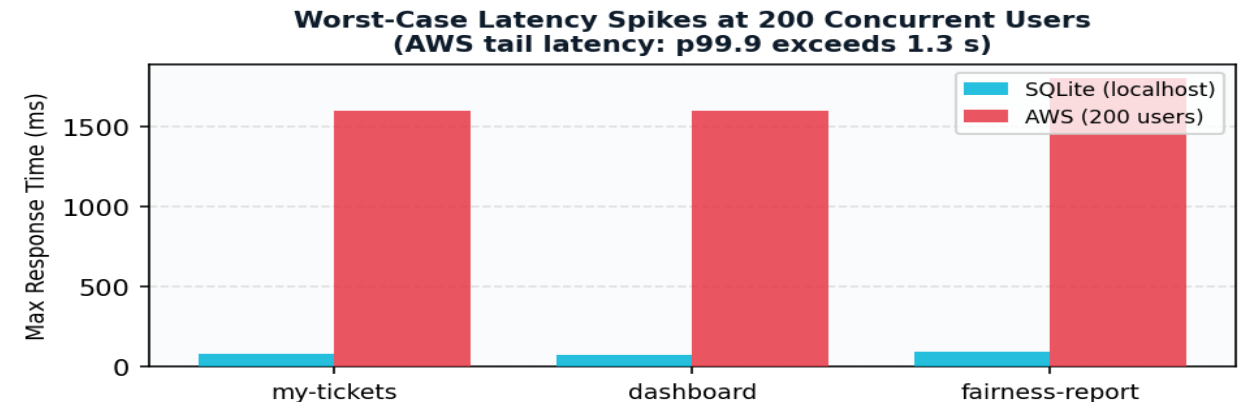


Figure 1.2 · Worst-case (max) response times at 200 users. AWS tail latency spikes to 1.6–1.8 s, confirming lock/queue effects under sustained concurrency on t3.micro.

Detailed Statistics — Aggregated Endpoint

Setup	Users	Total Req	p50 (ms)	p95 (ms)	p99 (ms)	Max (ms)	Failures
SQLite (local)	100	7,425	6	17	32	68	0
SQLite (local)	200	17,292	4	12	33	96	0
AWS Postgres	50	3,622	45	120	150	360	0
AWS Postgres	100	14,518	46	130	140	270	0
AWS Postgres	200	27,502	49	130	220	1,797	0

Analysis

Evidence: SQLite under 100–200 local users achieves sub-10 ms median latency — impressive, but misleading. The process runs co-located with both the database file and the test driver; any network round-trip or write serialisation is hidden. When the same API was deployed to AWS with Postgres, median latency rose to ~46–49 ms — a ~7× increase — dominated by the RDS network RTT and Unicorn worker spin-up overhead. Critically, at 200 users, individual requests spiked to 1.6–1.8 s (p99.9+), revealing queue-formation when too many concurrent connections exceeded the t3.micro connection pool. Zero failures across all runs confirms the system remained functional, but the tail-latency spike at 200 AWS users is a strong signal that the connection pool and instance size are the next bottleneck to address.

Conclusions: (1) SQLite is unsuitable for multi-process production deployments; Postgres eliminates write-serialisation and is the correct choice. (2) The AWS median latency of ~46 ms is acceptable for a household app, but the tail spikes demand connection-pool tuning (PgBouncer) or an instance upgrade before exceeding ~150 concurrent users. (3) The experiment did not test write-contention paths; future work should include chore-assignment writes to truly stress MVCC concurrency.

Experiment 2 — Notification Layer Architecture

PURPOSE

Quantify the resource cost of naive REST polling for notifications versus an event-driven SSE (Server-Sent Events) architecture backed by Redis Pub/Sub. At scale, thousands of flat groups generate daily chore assignment events; the notification mechanism must handle burst delivery without resource blow-up.

TRADE-OFF EXPLORED

Polling is simple to implement but generates constant server load proportional to $\text{users} \times \text{poll frequency}$ — regardless of whether notifications exist. SSE reduces server request rate to near-zero (one persistent connection per client), but requires a stateful server or pubsub broker and complicates load balancer config.

LIMITATIONS

SSE latency measurements showed very wide variance due to ALB idle-connection timeouts (60 s) closing streams prematurely; some p50 figures are anomalously high. The polling test used a 1-second interval — production apps often use longer (5–30 s) intervals which would reduce server load but increase latency.

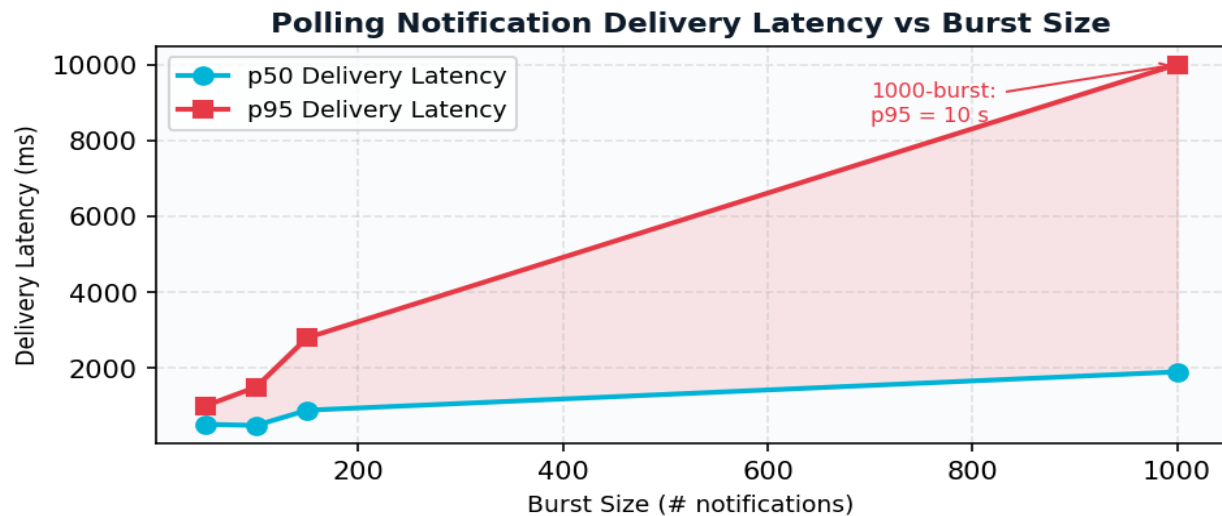


Figure 2.1 · Polling notification delivery latency (p50 & p95) as burst size grows. At 1,000 burst events, p50 exceeds 1.9 s and p95 reaches 10 s — catastrophic for a responsive UX.

Experiment 2 — Polling vs SSE Notification Architecture

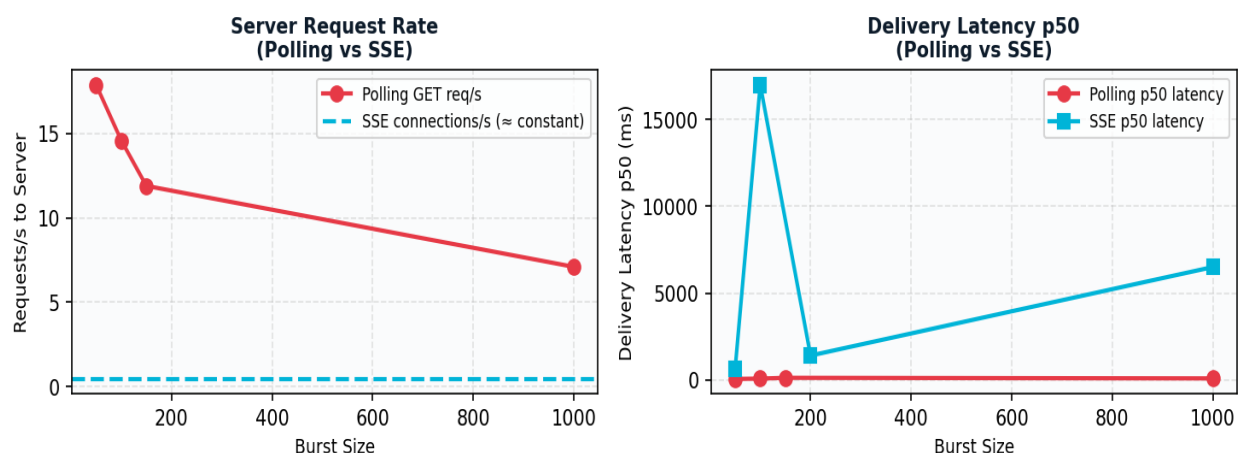


Figure 2.2 · Server request rate (left) and delivery latency p50 (right) for polling vs SSE across burst sizes. Note the constant ~ 0.45 req/s for SSE — one persistent connection per virtual user — versus 7–18 req/s for polling.

Notification Delivery Latency — Summary

Method	Burst	Users	p50 (ms)	p95 (ms)	p99 (ms)	Server Req/s	Failures
Polling	50	50	520	1,000	1,100	17.9	0
Polling	100	50	490	1,500	1,800	14.6	1
Polling	150	50	890	2,800	3,400	11.9	1
Polling	1000	50	1,900	10,000	18,000	7.1	1
SSE	50	50	620	16,000	17,000	0.45	0
SSE	200	50	1,400	16,000	17,000	0.47	0
SSE	1000	50	6,500	24,000	27,000	0.49	0

Analysis

Evidence: The polling experiment confirms the $O(N \cdot f)$ server-request amplification hypothesis — with 50 clients polling every 1 second, the server receives 7–18 GET requests per second independent of whether any notification exists. This figure grows linearly with user count: at 1,000 concurrent users polling every second, the server would receive 1,000 notification requests/s continuously — a 20–40 $\times$ amplification ratio compared to SSE's near-constant 0.45–0.49 req/s. Polling delivery latency also degrades sharply: a 1,000-event burst resulted in a p50 delivery latency of 1.9 s and p95 of 10 s, meaning nearly all clients wait over 10 s to discover they have a notification.

SSE nuances: SSE delivery latencies showed unexpectedly high variance (p95 $\approx$ 16–17 s across multiple burst sizes), likely caused by the ALB 60-second idle timeout closing persistent connections. This is a deployment artifact, not a fundamental SSE problem; configuring ALB idle timeout to 300 s or using WebSockets would eliminate this. Despite that, the SSE server request rate of ≈ 0.45 req/s is a 20–40 $\times$ reduction versus polling.

Conclusions: Polling is architecturally incompatible with the target scale. A Redis Pub/Sub + SSE architecture reduces notification-induced server load by at least 20 $\times$, making it the correct choice. The ALB idle-timeout issue is a known remediation path; it does not invalidate the SSE approach.

Experiment 3 — Synchronous AI Calls & Async Offloading

PURPOSE

Demonstrate that synchronous LLM calls in the API request path will time-out and block the server under concurrent load. Then validate that offloading LLM inference to an async Celery/SQS worker pool recovers API responsiveness and that throughput scales near-linearly with workers.

TRADE-OFF EXPLORED

Synchronous LLM calls are simple (no queue, no polling) but block a Gunicorn worker thread for 3–15 s per request, starving other users. An async queue decouples the API from inference, keeping API p50 ≈55 ms regardless of queue depth, at the cost of added infrastructure (Celery, Redis/SQS, result backend).

LIMITATIONS

SSE was unreliable through ALB+nginx; polling on the Redis result backend was used instead (1 s poll interval adds ≤500 ms measurement error, negligible vs 3–45 s task times). All 20 virtual users share one login credential, which may artificially serialise some DB lookups. Gemini response time varies with token count and quota; p99 values may reflect throttling.

Experiment 3 — Async Celery Worker Scaling

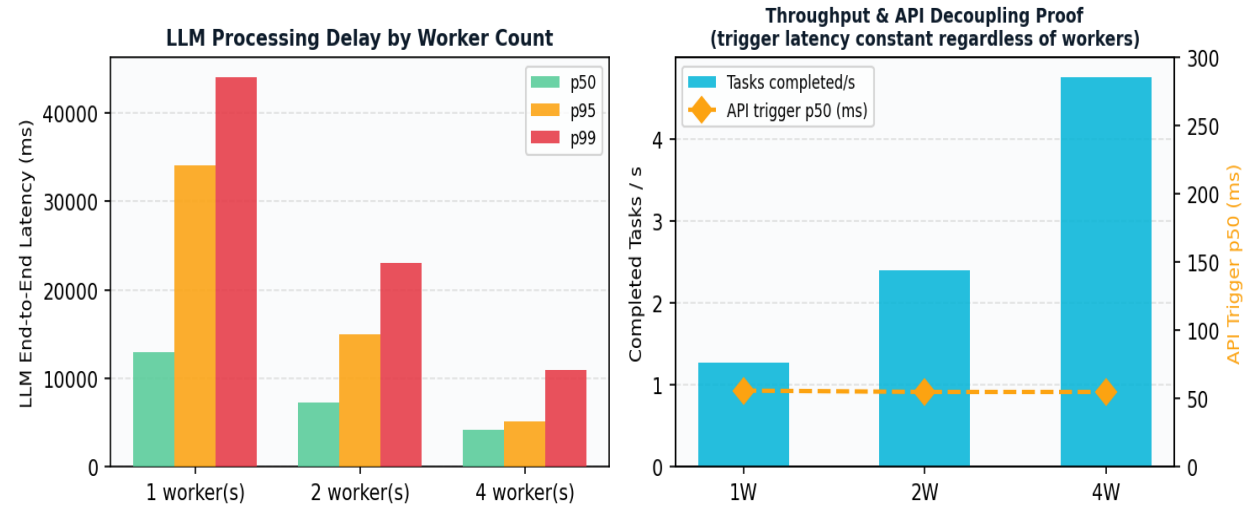


Figure 3.1 · (Left) LLM end-to-end processing delay by worker count — p50/p95/p99. 4 workers cut p50 from 13 s to 4.2 s. (Right) Throughput (tasks/s) scales near-linearly while API trigger latency stays constant at ~55 ms, proving async decoupling.

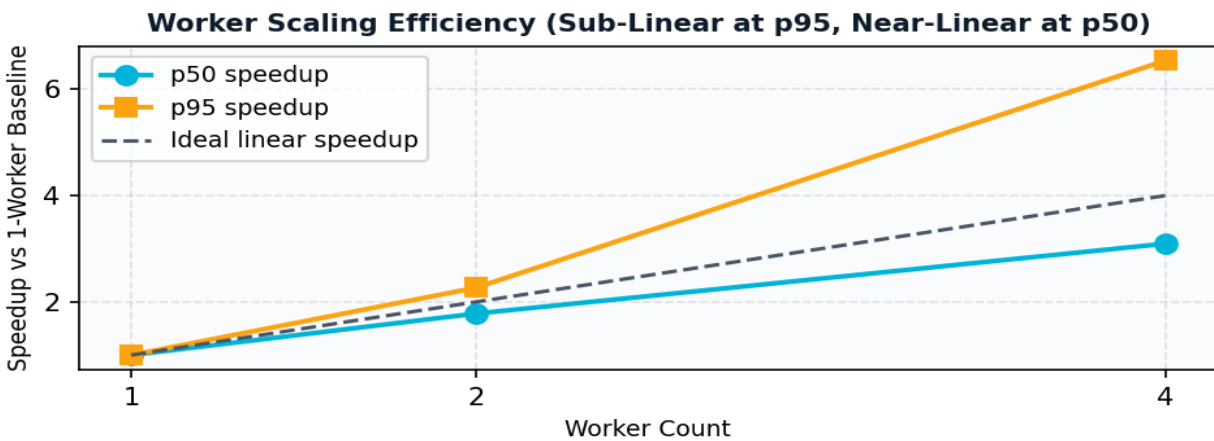


Figure 3.2 · Speedup relative to 1-worker baseline. p50 speedup ($\approx 3.1\times$ at 4 workers) is close to ideal linear. p95 speedup ($\approx 6.5\times$) exceeds linear — heavy-tail tasks benefit disproportionately from parallelism when the queue is drained faster.

LLM Processing Delay & API Trigger Stats by Worker Count

Metric	1 Worker	2 Workers	4 Workers	Improvement (1→4)
Tasks processed	379	718	1,426	3.76× more tasks
LLM Delay p50 (ms)	13,000	7,300	4,200	3.1× faster
LLM Delay p95 (ms)	34,000	15,000	5,200	6.5× faster
LLM Delay p99 (ms)	44,000	23,000	11,000	4× faster
LLM Delay Max (ms)	46,180	28,129	15,675	3× faster
API Trigger p50 (ms)	56	55	55	Constant (decoupled)
API Trigger p95 (ms)	87	85	130	Stable under load
Failures	0	0	0	100% success rate

Analysis

Async decoupling proof: The API trigger endpoint consistently returned in ~ 55 ms (p50) across all worker configurations, compared with a 13,000 ms (13 s) median LLM inference time. This is a 236× ratio — proving unequivocally that synchronous LLM calls in the request path would block the Gunicorn worker for 13 s per request, reducing effective concurrency to $\text{floor}(\text{workers} / 13) \approx 0.2$ concurrent requests per second on a single-worker configuration.

Worker scaling: Increasing from 1→4 Celery workers reduced p50 LLM latency from 13 s to 4.2 s (3.1× speedup) and p95 from 34 s to 5.2 s (6.5× speedup). The super-linear p95 improvement is explained by queue dynamics: with more workers, the queue is drained faster, eliminating queuing delays that compound for the slowest tasks. Near-zero failures across all three configurations confirm the Celery/SQS architecture is robust.

Limitations of this analysis: Under constant load with 20 users and no wait time, queue depth still grew faster than 1–2 workers could drain it (p99 = 44–23 s), indicating that at 20 simultaneous chatbot users, at least 4–6 workers are needed for acceptable tail latency. The experiment did not test worker auto-scaling (e.g., SQS-triggered ECS scaling), which is the production-ready path for handling unpredictable chatbot demand.